



STL

C++ 标准模板库STL的核心包括以下三个组件：

组件	描述
容器	容器是用来管理某一类对象的集合
算法	算法作用于容器。它们提供了执行各种操作的方式，包括对容器内容执行初始化、排序、搜索和转换等操作
迭代器	迭代器用于遍历对象集合的元素。这些集合可能是容器，也可能是容器的子集

vector（数组）

▼

```
1 #include<vector>
2 vector<int>vec;
```

vector常用内置函数

▼

```
1 //b为向量，将a中的元素和b中的元素整体交换
2 a.swap(b);
3
4 //b为向量，向量的比较操作还有 != >= > <= <
5 //如果vector里面的元素类型是简单类型（内置类型），可以直接使用“==”或者“!=”进行比较
6 //甚至可以使用“<=” “<” “>=” “>”比较两个vector大小：按照字典序排列
```

```
7 a == b;
```

用法	作用
<code>vec.begin(), vec.end()</code>	返回vector的首、尾 迭代器
<code>vec.front(), vec.back()</code>	返回vector的首、尾 元素
<code>vec.push_back()</code>	从vector末尾加入一个元素
<code>vec.size()</code>	返回vector当前的长度（大小）
<code>vec.pop_back()</code>	从vector末尾删除一个元素
<code>vec.empty()</code>	返回vector是否为空，1为空、0不为空
<code>vec.clear()</code>	清空vector
<code>vec.resize()</code>	重新设置容器大小

algorithm对vector常用函数

```
▼
1  #include<algorithm>
2
3  //对a中的从a.begin()（包括它）到a.end()（不包括它）的元素进行从小到大排列
4  sort(a.begin(),a.end());
5
6  //对a中的从a.begin()（包括它）到a.end()（不包括它）的元素倒置，但不排列，如a中元素为
   1,3,2,4,倒置后为4,2,3,1
7  reverse(a.begin(),a.end());
8
9  //把a中的从a.begin()（包括它）到a.end()（不包括它）的元素复制到b中，从b.begin()+1的位
   置（包括它）开始复制，覆盖掉原有元素
10 copy(a.begin(),a.end(),b.begin()+1);
11
12 //在a中的从a.begin()（包括它）到a.end()（不包括它）的元素中查找10，若存在返回其在向量中的
   位置
13 find(a.begin(),a.end(),10);
14
15 //在a中的从a.begin()（包括它）到a.end()（不包括它）的元素中查找10的个数，返回个数
16 count(a.begin(), a.end(),10);
```

下标只能用来获取已经存在的元素,不能通过下标赋值

List（双向链表）

▼

```
1 #include<list>
2 list<int>l;
```

方法	描述
<code>l.insert()</code>	将新元素插入到迭代器指向的位置之前
<code>l.push_back()</code>	在容器的末尾添加了一个新元素
<code>l.push_front()</code>	在前面增加了一个新元素
<code>l.pop_back()</code>	删除最后一个元素
<code>l.pop_front()</code>	删除第一个元素
<code>l.empty()</code>	列表是否为空
<code>l.size()</code>	查找列表中存在的元素数
<code>l.front()</code>	返回列表的第一个元素
<code>l.back()</code>	返回列表的最后一个元素
<code>l.resize()</code>	更改列表容器的大小

stack(栈)

▼

```
1 #include<stack>
2 stack<int>s;
```

用法	作用
----	----

<code>st.top()</code>	返回stack的栈顶 元素
<code>st.push()</code>	从stack栈顶加入一个元素
<code>st.size()</code>	返回stack当前的长度（大小）
<code>st.pop()</code>	从stack栈顶弹出一个元素
<code>st.empty()</code>	返回stack是否为空，1为空、0不为空

queue（队列）

▼

```
1 #include<queue>
2 queue<int>q;
```

用法	作用
<code>q.front(), q.back()</code>	返回queue的首、尾 元素
<code>q.push()</code>	从queue末尾加入一个元素
<code>q.size()</code>	返回queue当前的长度（大小）
<code>q.pop()</code>	从queue末尾删除一个元素
<code>q.empty()</code>	返回queue是否为空，1为空、0不为空

priority_queue（优先队列）

▼

```
1 #include<queue>
2 priority_queue<int>q;
```

用法	作用
<code>q.top()</code>	返回priority_queue的首 元素

<code>q.push()</code>	向priority_queue中加入一个元素
<code>q.size()</code>	返回priority_queue当前的长度（大小）
<code>q.pop()</code>	从priority_queue末尾删除一个元素
<code>q.empty()</code>	返回priority_queue是否为空，1为空、0不为空

deque（双端队列）

▼

```
1 #include<queue>
2 deque<int>q;
```

用法	作用
<code>q.begin(), q.end()</code>	返回deque的首、尾 迭代器
<code>q.front(), q.back()</code>	返回deque的首、尾 元素
<code>q.push_back()</code>	从队尾入队一个元素
<code>q.push_front()</code>	从队头入队一个元素
<code>q.pop_back()</code>	从队尾出队一个元素
<code>q.pop_front()</code>	从队头出队一个元素
<code>q.clear()</code>	清空队列

除了这些用法之外，deque比queue更优秀的一个性质是它支持随机访问，即可以像数组下标一样取出其中的一个元素。

即：`q[i]`。

set（集合）

声明：

▼

```
1 #include<set>
2 set<int> s;
3 set<char> s;
4 set<pair<int,int> > s;
5 set<node> s;
6 struct node{...};
```

用法	作用
<code>s.begin(), s.end()</code>	返回set的首、尾迭代器
<code>s.erase()</code>	删除集合中元素
<code>s.find()</code>	集合中指向要寻找元素的迭代器，不存在这个元素，就返回 <code>s.end()</code>
<code>s.size();</code>	当前集合的元素个数
<code>s.empty()</code>	判断当前集合是否为空
<code>s.clear()</code>	清空集合
<code>s.insert()</code>	向集合中加入元素

集合	底层实现	是否有序	数值是否可以重复	能否更改数值	查询效率	增删效率
std::set	红黑树	有序	否	否	O(log n)	O(log n)
std::multiset	红黑树	有序	是	否	O(logn)	O(logn)
std::unordered_set	哈希表	无序	否	否	O(1)	O(1)

map（映射）

声明：

▼

```
1 #include<map>
```

```
2 map<int, char> mp;
```

用法	作用
<code>m.begin(), m.end()</code>	返回map的首、尾 迭代器
<code>m.count()</code>	获取与给定键匹配的元素数
<code>m.find()</code>	搜索具有给定键的元素，不存在这个元素，就返回 <code>m.end()</code>
<code>m.size();</code>	返回map中的元素数
<code>m.empty()</code>	判断map是否为空
<code>m.clear()</code>	删除map的所有元素
<code>m.insert()</code>	向map中插入元素

映射	底层实现	是否有序	数值是否可以重复	能否更改数值	查询效率	增删效率
std::map	红黑树	key有序	key不可重复	key不可修改	O(logn)	O(logn)
std::multimap	红黑树	key有序	key可重复	key不可修改	O(log n)	O(log n)
std::unordered_map	哈希表	key无序	key不可重复	key不可修改	O(1)	O(1)